

BÀI 3. THỰC HÀNH SPRING BOOT – TẠO DỰ ÁN SPRING BOOT CÓ KẾT NỐI CƠ SỞ DỮ LIỆU

Yêu cầu: xây dựng Spring Boot project

Thời lượng: 6 tiết

Mục tiêu:

- Tạo được ứng dụng Java dùng Spring Boot
- Kết nối được cơ sở dữ liệu cho ứng dụng dùng Spring Boot

Nội dung:

Câu 1. Tạo một dự án Spring MVC đầu tiên

a/ Tạo Databaes

```
CREATE DATABASE WebStore;
```

```
GO
```

```
USE WebStore;
```

```
GO
```

b/ Tạo Project Spring Mvc

Truy cập vào link <https://start.spring.io/> để tạo project Spring Mvc

Sau khi truy cập vào link tạo project theo cấu hình như sau

Project	Maven
Language	Java
Spring Boot	3.4.0
Group	WebStore
Artiface	WebStore
Name	WebStore
Package name	webapp
Packing	jar

Java	22
Dependencies	Spring Boot DevTools
	Spring Web
	Thymeleaf
	Spring Data JPA
	MS SQL Server Driver

c/ Configuration database

Vào src/main/resources/application.properties, thêm vào nội dung sau để khai báo kết nối đến CSDL SQL Server

```
spring.jpa.hibernate.ddl-auto=update
spring.datasource.url=jdbc:sqlserver://localhost:1433;databaseName=WebStore;TrustServerCertificate=true;
spring.datasource.username=sa
spring.datasource.password=123
spring.datasource.driver-class-name=com.microsoft.sqlserver.jdbc.SQLServerDriver
spring.jpa.show-sql: true
spring.jpa.hibernate.naming.implicit-
strategy=org.hibernate.boot.model.naming.ImplicitNamingStrategyLegacyJpaImpl
spring.jpa.hibernate.naming.physical-
strategy=org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl
```

Lưu ý trong đoạn code trên ta kết nối đến Sql Server với username là sa và password=123, tên CSDL là WebStore

d/ Tạo HomeController

Trong thư mục **main/java/webapp** tạo thêm 2 thư mục là **controllers** và **models**

Sau đó hãy tạo **HomeController** với nội dung như sau:

```
@Controller
public class HomeController {
    @GetMapping("/")
    public String index() {
        return "home/index";
    }
}
```

e/ Tạo view cho action index trong controller HomeController

Vào thư mục **main/resoures/templates** tạo thư mục **home** và file **index.html** trong thư mục home

Trong file **index.html** hãy đặt nội dung bất kỳ ví dụ

```
<h2>Welcome Ladies and Gentlemen</h2>
```

f/ Chạy thử dự án

Nhấp phải vào tên của Project trong cửa sổ Project Explorer, chọn Run As -> Spring Boot App. Sau khi chạy xong, hãy mở trình duyệt web và truy cập **localhost:8080/** để xem kết quả.

Câu 2. Tạo một dự án Spring Boot bằng Eclipse có khai báo kết nối cơ sở dữ liệu MySQL

a/ Cài đặt Spring Tool cho Eclipse: Help -> Eclipse Marketplace, gõ vào ô Find từ khóa tìm kiếm “Spring” và nhấn phím Enter.

Chọn cài đặt Spring Tool 4 và thực hiện đủ các bước cài đặt.

b/ Tạo Spring Boot Project mới:

- Mở Eclipse, chọn menu File ->New -> Other...
- Mở rộng mục Spring Boot, chọn Spring Starter Project, nhập Next.
Thiết lập metadata như sau:

New Spring Starter Project

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

Description:

Package:

Working sets

☐ Add project to working sets

Working sets:

- Nhấp nút Next
- Lựa chọn các thành phần phụ thuộc như trong hình sau:

New Spring Starter Project Dependencies

Spring Boot Version:

Frequently Used:

☒ MySQL Driver ☒ Spring Boot DevTools ☒ Spring Data JPA

☒ Spring Web ☒ Thymeleaf

Available:

Type to search dependencies

Selected:

X Spring Boot DevTools

X Spring Data JPA

X MySQL Driver

X Thymeleaf

X Spring Web

- Nhấp Next, Finish.

c/ Tạo cơ sở dữ liệu WebStore trên MySQL

```
CREATE DATABASE WebStore;
```

```
USE WebStore;
```

d/ Khai báo cấu hình kết nối cơ sở dữ liệu cho Spring boot project tại file application.properties

```
spring.jpa.hibernate.ddl-auto=update
spring.datasource.url=jdbc:mysql://localhost:3306/WebStore
spring.datasource.username=root
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.show-sql: true
spring.jpa.hibernate.naming.implicit-
strategy=org.hibernate.boot.model.naming.ImplicitNamingStrategyLegacyJpaImpl
spring.jpa.hibernate.naming.physical-
strategy=org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl
```

e/ Tạo HomeController

Trong thư mục **main/java/webapp** tạo thêm 2 thư mục là **controllers** và **models**

Sau đó ta tạo HomeController với nội dung như sau:

```
@Controller
public class HomeController {
    @GetMapping("/webstore")
    public String index() {
        return "home/index";
    }
}
```

f/ Tạo view cho action index trong controller HomeController

Vào thư mục **main/resoures/templates** tạo thư mục **home** và file index.html trong thư mục home với nội dung tự biên tập.

g/ Chạy thử dự án

Nhấp phải vào tên của Project trong cửa sổ Project Explorer, chọn Run As -> Spring Boot App. Sau khi chạy xong, hãy mở trình duyệt web và truy cập *localhost:8080/webstore* để xem kết quả.

Câu 3: Thực hành các hoạt động quản lý dữ liệu với CRUD (Create, Read, Update, Delete)

a/ Tạo bảng Category

Sử dụng lệnh sau để tạo bảng Category như sau

```
CREATE TABLE Category(  
    CategoryId SMALLINT NOT NULL PRIMARY KEY AUTO_INCREMENT,  
    CategoryName VARCHAR(64) NOT NULL  
);  
INSERT INTO Category (CategoryId, CategoryName) VALUES  
    (1, 'Men shoes'),  
    (2, 'Women shoes'),  
    (3, 'Hand bag'),  
    (4, 'fashion accessories');
```

b/ Tạo Model

Tạo class Category

Vào thư mục models trong java/webapp tạo Category.java và viết code như sau

```
@Entity(name = "Category")  
public class Category {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    @Column(name = "CategoryId")  
    private short id;  
    @Column(name = "CategoryName")  
    private String name;
```

```

public Category(){
}
public Category(short id, String name){
    this.id = id;
    this.name = name;
}
public short getId() {
    return id;
}
public String getName() {
    return name;
}
public void setId(short id) {
    this.id = id;
}
public void setName(String name) {
    this.name = name;
}
}

```

c/ Tạo interface CategoryRepository

Vào models tạo tiếp CategoryRepository.java viết code như sau

```

public interface CategoryRepository extends CrudRepository<Category,Short> {
}

```

d/ Tạo CategoryController

Trong thư mục controllers ta tạo file CategoryController.java như sau

@Controller

```
public class CategoryController {
```

```
    @Autowired
```

```
    CategoryRepository repository;
```

```
    @GetMapping("/category")
```

```
    public String index(Model model){
```

```
        model.addAttribute("list", repository.findAll());
```

```
        return "category/index";
```

```
    }
```

```
    @GetMapping("/category/delete/{id}")
```

```
    public String delete(@PathVariable("id") short id){
```

```
        repository.deleteById(id);
```

```
        return "redirect:/category";
```

```
    }
```

```
    @GetMapping("/category/add")
```

```
    public String add(){
```

```
        return "category/add";
```

```
    }
```

```
    @PostMapping("/category/add")
```

```
    public String add(Category obj){
```

```
        repository.save(obj);
```

```
        return "redirect:/category";
```

```
    }
```

```
    @GetMapping("/category/edit/{id}")
```

```
    public String edit(@PathVariable("id") short id, Model model){
```

```
        model.addAttribute("o", repository.findById(id).get());
```

```

        return "category/edit";
    }

    @PostMapping("/category/edit/{id}")
    public String edit(@PathVariable("id") short id, Category obj){
        repository.save(obj);
        return "redirect:/category";
    }
}

```

e/ Tạo View cho CategoryController

View index.html

Trong thư mục **templates**, tạo thư mục category, rồi tạo index.html với nội dung sau:

```

<a href="/category/add">Add</a>
<table>
    <thead>
        <tr>
            <th>Id</th>
            <th>Name</th>
            <th>Command</th>
        </tr>
    </thead>
    <tbody>
        <tr th:each="item : ${list}">
            <td th:text="${item.id}"></td>
            <td th:text="${item.name}"></td>
            <td>
                <a th:href="@{/category/edit/{id}(id=${item.id})}">Edit</a>
                <a th:href="@{/category/delete/{id}(id=${item.id})}">Delete</a>
            </td>
        </tr>
    </tbody>
</table>

```

```

        </td>
    </tr>
</tbody>
</table>

```

f/ View add.html

Trong templates/category tạo tiếp add.html, rồi nhập nội dung sau:

```

<form method="post">
    <div>
        <label for="name">Name</label>
        <div>
            <input type="text" name="name" id="name">
        </div>
    </div>
    <div>
        <button>Save Changes</button>
    </div>
</form>

```

g/ View edit.html

```

<form method="post">
    <div>
        <label for="name">Name</label>
        <div>
            <input type="text" name="name" id="name" th:value="${o.name}">
        </div>
    </div>
    <div>

```

```

        <button>Save Changes</button>

    </div>

</form>

```

Câu 4: Upload File

a/ Cấu hình upload file

Để upload vào thư mục đang chạy web ta cần tạo cấu hình chuyển đổi upload file trên thư mục web đang chạy trên server bằng cách tạo **MvcConfig.java** trong **webapp** như sau:

```

@Configuration
public class MvcConfig implements WebMvcConfigurer{

    @Override

    public void addResourceHandlers(ResourceHandlerRegistry registry) {

        String dirName = "images";

        Path dir = Paths.get("images");

        String uploadPath = dir.toFile().getAbsolutePath();

        registry.addResourceHandler("/") + dirName + "/*").addResourceLocations("file://"
+ uploadPath + "/");

    }

}

```

b/ Tạo Service Upload File

Tạo thêm thư mục **services** trong **src/main/java/webapp**

Tạo thư mục **images** trong **src/main/resources/static**

Tạo class StorageProperties

Tạo **StorageProperties.java** trong thư mục services để khai báo đường dẫn images trên thư mục web resource như sau:

```

@ConfigurationProperties("storage")

public class StorageProperties {

```

```

private String location = "./src/main/resources/static/images";

public String getLocation() {
    return location;
}

public void setLocation(String location) {
    this.location = location;
}
}

```

Sau khi tạo class `StorageProperties` xong ta cấu hình chạy khi webapp được khởi tạo trong class **`WebStoreApplication`** như sau:

```

@EnableConfigurationProperties(StorageProperties.class)
public class WebStoreApplication {

    public static void main(String[] args) {
        SpringApplication.run(WebStoreApplication.class, args);
    }
}

```

Trong code trên ta chỉ thêm phần code **`@EnableConfigurationProperties(StorageProperties.class)`**, để biết được đường dẫn lưu trữ file trong resources

c/ Tạo class Helper

Tạo `Helper.java` trong thư mục `services` để xử lý upload file với nội dung sau

```

public class Helper {

    public static String saveFile(String uploadDir, MultipartFile file, int len) throws IOException {
        Path path = Paths.get(uploadDir);
        if(!Files.exists(path)) {
            Files.createDirectories(path);

```

```

    }

    String name = file.getOriginalFilename();
    String ext = name.substring(name.lastIndexOf('.'));
    String fileName = randomString(len - ext.length()) + ext;
    Path filePath = path.resolve(fileName);
    file.transferTo(filePath);
    return fileName;
}

public static String randomString(int len){
    StringBuilder sb = new StringBuilder(len);
    String pattern = "1234567890qwertyuiopasdfghjklzxcvbnm";
    Random random = new Random();
    for(int i = 0; i < len; i++){
        sb.append(pattern.charAt(random.nextInt(pattern.length())));
    }
    return sb.toString();
}
}
}

```

Trong class trên có phương thức **randomString** để tạo chuỗi ngẫu nhiên với chiều dài truyền vào là **len**

Đối với phương thức **saveFile** xử lý với mục đích lưu trữ vào thư mục, trong quá trình lưu file cần tạo chuỗi ngẫu nhiên nhằm tránh trường hợp trùng tên tập tin, sẽ bị ghi đè

d/ Cấu hình kích thước file upload

Trường hợp kích thước file vượt quá mặc định cho phép trong Spring MVC ta cần cấu hình kích thước tập tin tối đa có thể upload trong application.properties như sau

```

spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=10MB

```

e/ Tạo model

Tạo class Image

Tạo Image.java trong thư mục models để lưu trữ các thông tin như đường dẫn, kiểu tập tin, kích thước như trong code sau:

```
@Entity
public class Image {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    @Column(name = "ImageId")
    private int id;
    @Column(name = "ImageUrl")
    private String url;
    private long size;
    private String type;
    public Image() {
    }
    public Image(int id, String url, long size, String type) {
        super();
        this.id = id;
        this.url = url;
        this.size = size;
        this.type = type;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
}
```

```

    }

    public String getUrl() {
        return url;
    }

    public void setUrl(String url) {
        this.url = url;
    }

    public long getSize() {
        return size;
    }

    public void setSize(long size) {
        this.size = size;
    }

    public String getType() {
        return type;
    }

    public void setType(String type) {
        this.type = type;
    }
}

```

Class ImageRepository

Tạo tiếp ImageRepository.java trong models với nội dung sau:

```

public interface ImageRepository extends CrudRepository<Image, Integer>{
}

```

f/ Tạo Controller

Tạo UploadController

Trong thư mục controllers tạo UploadController.java với nội dung sau:

```
@Controller
public class UploadController {

    @Autowired
    private ImageRepository repository;

    @GetMapping("/upload")
    public String index(Model model){
        model.addAttribute("list", repository.findAll());
        return "upload/index";
    }

    @GetMapping("/upload/add")
    public String add(){
        return "upload/add";
    }

    @PostMapping("/upload/add")
    public String add(MultipartFile f){
        try {
            String fileName = Helper.saveFile("images", f, 32);
            Image image = new Image(0, fileName, f.getSize(), f.getContentType());
            repository.save(image);
            return "redirect:/upload";
        } catch (IOException e) {
            e.printStackTrace();
        }
        return "upload/add";
    }
}
```

}

g/ Tạo Views

Tạo thư mục **upload** trong **templates**

Tạo view index

Trong thư mục upload tạo index.html với nội dung sau để hiển thị thông tin upload file như sau:

```

<a href="/upload/add">Add</a>

<table>

  <tbody>

    <tr th:each="o : ${list}">

      <td th:text="${o.id}"></td>

      <td>

      </td>

      <td th:text="${o.type}"></td>

    </tr>

  </tbody>

</table>

```

Tạo view add

Trong thư mục upload tiếp tục tạo add.html để upload file

```

<form enctype="multipart/form-data" method="post">

  <div>

    <label for="f">Image</label>

    <div>

      <input type="file" name="f" id="f" accept="image/*">

    </div>

  </div>

</div>

```

```
<div>

    <button>Upload</button>

</div>

</form>
```

---Hết bài thực hành 3---

BÀI 4. THỰC HÀNH TẠO WEBSITE SPRING BOOT

CÓ KẾT NỐI CƠ SỞ DỮ LIỆU

Yêu cầu: xây dựng lại website HandmadeLeather

Thời lượng: 6 tiết

Mục tiêu:

- Tạo được website dùng Spring Boot
- Kết nối được cơ sở dữ liệu, trình bày dữ liệu lên View sử dụng Thymeleaf

Nội dung:

Chuẩn bị:

Cơ sở dữ liệu: sử dụng lại cơ sở dữ liệu “sell” có table hanghoa như sau:

Field Name	Data Type	Key
mahh	varchar(255)	Primary
tenhh	varchar(255)	
dongia	int(11)	
xuatxu	varchar(255)	
loaihh	int(11)	
hinh	varchar(255)	
xuhuong	boolean	
phobien	boolean	

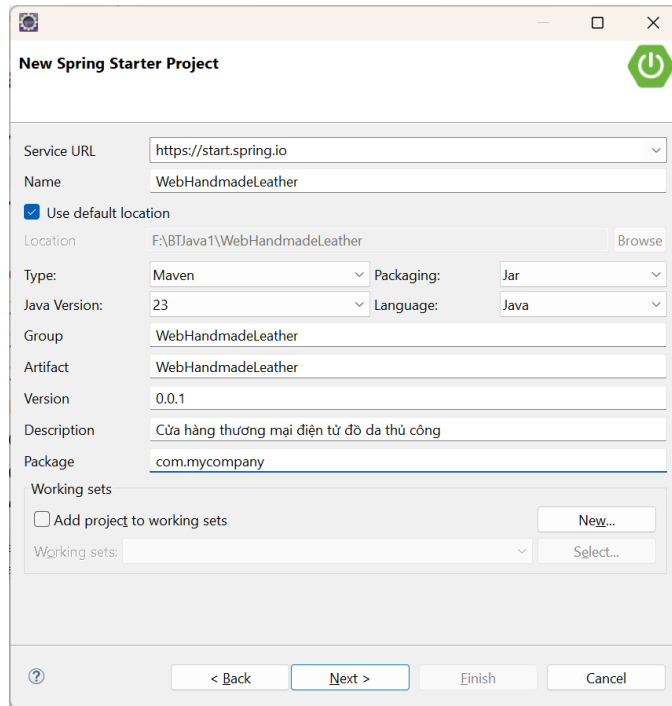
Hình ảnh: sao chép thư mục Images của bài thực hành 2 vào thư mục **static** của project sẽ được tạo ở câu 1.

Các file CSS: sao chép thư mục css của bài thực hành 2 vào thư mục static của project sẽ được tạo ở câu 1.

Admin Template: tải thư mục admin_template giảng viên đã gửi trên group lớp, các file html thì sao chép vào thư mục **admin** trong thư mục **templates**. Các file css, js, image thì sao chép vào thư mục tương ứng trong thư mục **static**.

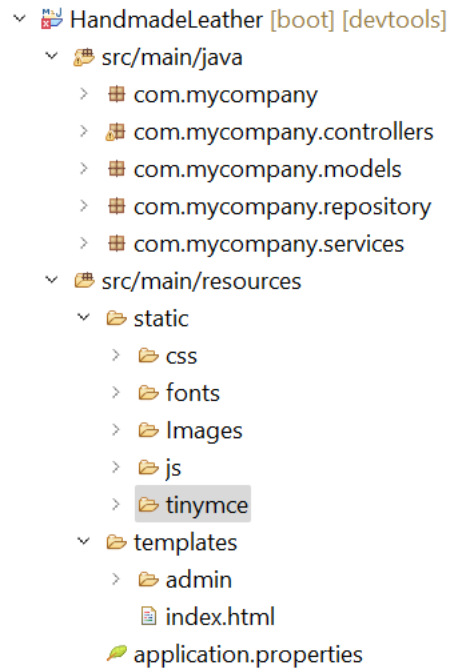
Câu 1. Tạo website HandmadeLeather

a/ Tạo một dự án Spring Boot



The screenshot shows the 'New Spring Starter Project' dialog box. The 'Service URL' is set to 'https://start.spring.io'. The 'Name' is 'WebHandmadeLeather'. The 'Location' is 'F:\BTJava1\WebHandmadeLeather'. The 'Type' is 'Maven', 'Packaging' is 'Jar', 'Java Version' is '23', and 'Language' is 'Java'. The 'Group' is 'WebHandmadeLeather', 'Artifact' is 'WebHandmadeLeather', 'Version' is '0.0.1', 'Description' is 'Cửa hàng thương mại điện tử đồ da thủ công', and 'Package' is 'com.mycompany'. The 'Add project to working sets' checkbox is unchecked. The 'Next >' button is highlighted.

b/ Tạo cấu trúc dự án với các package và folder như hình sau:



c/ Trong thư mục **templates**, tạo file index.html. Sử dụng lại mã nguồn của file index.jsp trong bài thực hành 2 và bỏ đi các thẻ jsp; sau đó cập nhật lại một số thẻ như sau:

```

1 <!DOCTYPE html>
2 <html xmlns:th="http://www.thymeleaf.org">
3 <head>
4     <meta charset="utf-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1">
6     <title>Đồ da thủ công - Trang chủ</title>
7     <link rel="stylesheet" th:href="@{/css/user_index_style.css}">
8 </head>

```

```

32 

```

```

40 <section class="trending">
41 <h2>TRENDING PRODUCTS</h2>
42 <p class="intro">Technology increasingly dominates the fashion industry. However, tradition
43
44 <div th:each="product:${listProducts}">
45     <div class="product">
46         
47         <p th:text="${product.tenhh}"></p> <p th:text="${product.dongia}"></p>
48         <button>Add to cart</button>
49     </div>
50 </div>
51 <hr/>
52 </section>

```

```

<section class="popular">
  <h2>POPULAR PRODUCTS</h2>
  <p class="intro">Handmade leather products are popular and widely used. In which, me
<div th:each="product:${listProPhobien}">
  <div class="product">
    
      <p th:text="${product.tenhh}"></p> <p th:text="${product.dongia}"></p>
    <button>Add to cart</button>
  </div>
</div>
<hr>
</section>

```

d/ Trong package com.mycompany.model, tạo entity class tên là Product như sau:

```

package com.mycompany.models;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name="hanghoa")
public class Product {

    @Id
    @Column(name="mahh")
    private String mahh;

    @Column(name="tenhh")
    private String tenhh;

    @Column(name="dongia")
    private int dongia;

    @Column(name="xuatxu")
    private String xuatxu;

    @Column(name="loaihh")

```

```
private int loaihh;  
@Column(name="hinh")  
private String hinh;  
@Column(name="xuhuong")  
private boolean xuhuong;  
@Column(name="phobien")  
private boolean phobien;  
public Product() {  
    }  
  
    public Product(String mahh, String tenhh, int dongia, String xuatxu, int loaihh,  
String hinh, boolean xuhuong,  
                boolean phobien) {  
    super();  
    this.mahh = mahh;  
    this.tenhh = tenhh;  
    this.dongia = dongia;  
    this.xuatxu = xuatxu;  
    this.loaihh = loaihh;  
    this.hinh = hinh;  
    this.xuhuong = xuhuong;  
    this.phobien = phobien;  
    }  
  
    public String getMahh() {  
        return mahh;  
    }  
  
    public void setMahh(String mahh) {  
        this.mahh = mahh;
```

```
}  
  
public String getTenhh() {  
    return tenhh;  
}  
  
public void setTenhh(String tenhh) {  
    this.tenhh = tenhh;  
}  
  
public int getDongia() {  
    return dongia;  
}  
  
public void setDongia(int dongia) {  
    this.dongia = dongia;  
}  
  
public String getXuatxu() {  
    return xuatxu;  
}  
  
public void setXuatxu(String xuatxu) {  
    this.xuatxu = xuatxu;  
}  
  
public int getLoaihh() {  
    return loaihh;  
}  
  
public void setLoaihh(int loaihh) {  
    this.loaihh = loaihh;  
}  
  
public String getHinh() {  
    return hinh;
```

```

    }

    public void setHinh(String hinh) {
        this.hinh = hinh;
    }

    public boolean getXuhuong() {
        return xuhuong;
    }

    public void setXuhuong(boolean xuhuong) {
        this.xuhuong = xuhuong;
    }

    public boolean getPhobien() {
        return phobien;
    }

    public void setPhobien(boolean phobien) {
        this.phobien = phobien;
    }
}

```

e/ Trong package `com.mycompany.repository`, tạo interface **ProductRepository**

```

package com.mycompany.repository;

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;

import com.mycompany.models.*;

public interface ProductRepository extends JpaRepository<Product,String>{

    List<Product> findByxuhuong(boolean xuhuong);

    List<Product> findByphobien(boolean phobien);

}

```

f/ Trong package `com.mycompany.service`, tạo interface `ProductService`

```
package com.mycompany.services;

import java.util.List;
import com.mycompany.models.Product;

public interface ProductService {
    List<Product> getAll();
    List<Product> findByxuhuong(boolean xuhuong);
    List<Product> findByphobien(boolean phobien);
    Product getProductById(String id);
    Boolean create(Product product);
    Boolean update(Product product);
    Boolean delete(Product product);
}
```

g/ Trong package `com.mycompany.services`, tạo tiếp một class tên `ProductServiceImpl` là implements của interface `ProductService`

```
package com.mycompany.services;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.mycompany.models.Product;
import com.mycompany.repository.ProductRepository;

@Service
public class ProductServiceImpl implements ProductService{
    @Autowired
```

```
private ProductRepository productRepository;

@Override

public List<Product> getAll() {

    // TODO Auto-generated method stub

    return this.productRepository.findAll();

}

@Override

public List<Product> findByxuhuong(boolean xuhuong){

    return this.productRepository.findByxuhuong(xuhuong);

}

@Override

public List<Product> findByphobien(boolean phobien){

    return this.productRepository.findByphobien(phobien);

}

@Override

public Product getProductById(String id) {

    // TODO Auto-generated method stub

    return null;

}

@Override

public Boolean create(Product product) {

    // TODO Auto-generated method stub

    try {

        this.productRepository.save(product);

        return null;

    } catch (Exception e) {
```

```

        e.printStackTrace();
    }
    return false;
}
@Override
public Boolean update(Product product) {
    // TODO Auto-generated method stub
    return null;
}
@Override
public Boolean delete(Product product) {
    // TODO Auto-generated method stub
    return null;
}
}

```

h/ Trong package com.mycompany.controllers, tạo class **HomeController**

```

package com.mycompany.controllers;

import java.util.List;

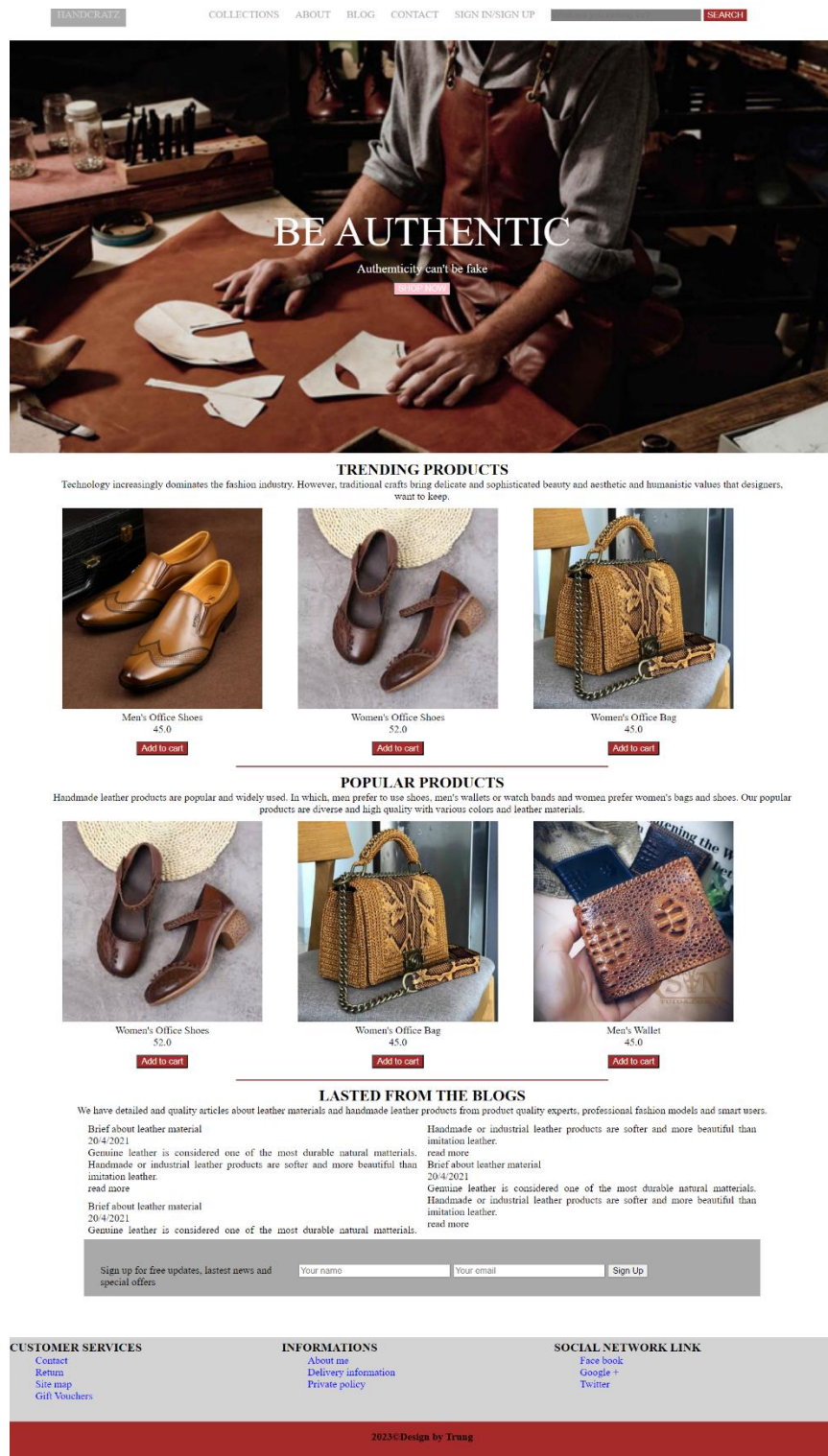
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestMapping;
import com.mycompany.models.Product;
import com.mycompany.services.ProductService;

```

@Controller

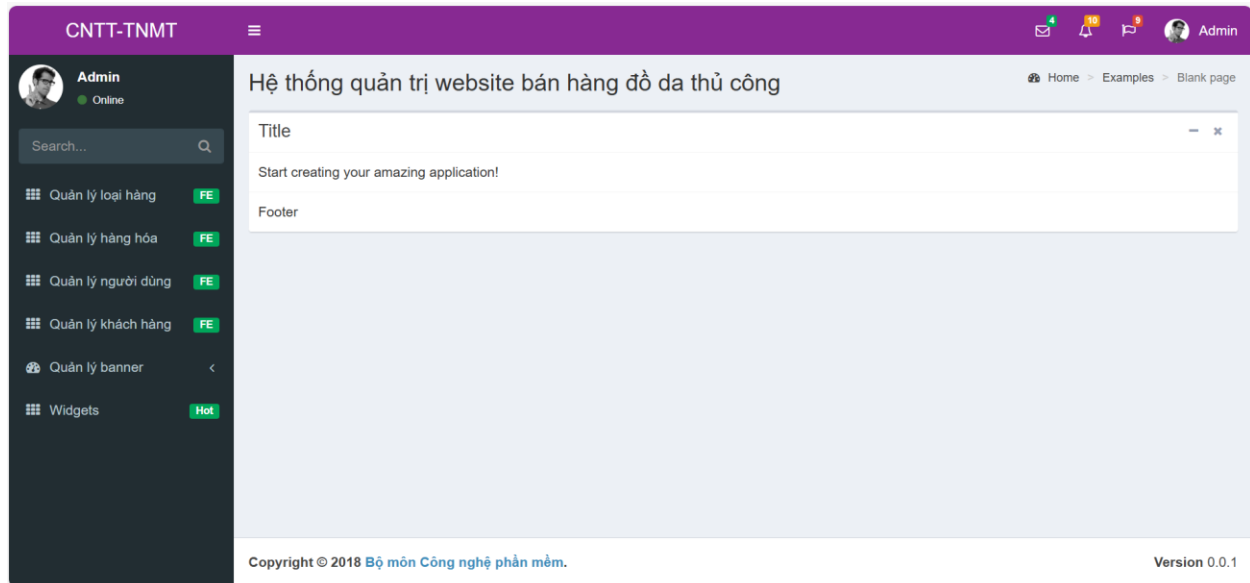
```
public class HomeController {  
    @Autowired  
    private ProductService productService;  
    @RequestMapping("/Home")  
    public String home(Model model) {  
        List<Product> lstPro=this.productService.findByxuhuong(true);  
        List<Product> lstProPhobien=this.productService.findByphobien(true);  
        model.addAttribute("listPro",lstPro);  
        model.addAttribute("listProPhobien", lstProPhobien);  
  
        return "index";  
    }  
}
```

Kết quả dự kiến của câu 1:



Câu 2. Tải admin_templates và thêm vào project để tạo nhóm giao diện phía admin.

a/ Cập nhật lại file index.html để đạt được giao diện tương tự như sau:



Gợi ý chỉnh sửa:

```

1 <!DOCTYPE html>
2 <html xmlns:th="http://www.thymeleaf.org">
3 <head>
4   <meta charset="utf-8">
5   <meta http-equiv="X-UA-Compatible" content="IE=edge">
6   <title>Admin - Home</title>
7   <!-- Tell the browser to be responsive to screen width -->
8   <meta content="width=device-width, initial-scale=1, maximum-scale=1, user-scalable=no">
9   <!-- Bootstrap 3.3.7 -->
10  <link rel="stylesheet" th:href="@{/css/bootstrap.min.css}">
11  <link rel="stylesheet" th:href="@{/css/font-awesome.min.css}">
12  <link rel="stylesheet" th:href="@{/css/AdminLTE.css}">
13  <link rel="stylesheet" th:href="@{/css/_all-skins.min.css}">
14  <link rel="stylesheet" th:href="@{/css/jquery-ui.css}">
15  <link rel="stylesheet" th:href="@{/css/style.css}" />
16  <script th:src="@{/js/angular.min.js}"></script>
17  <script th:src="@{/js/app.js}"></script>

```

```

57   
216     <i class="fa fa-th"></i> <span>Quản lý hàng hóa </span>
217     <span class="pull-right-container">
218       <small class="label pull-right bg-green">FE</small>
219     </span>
220   </a>

```

Sinh viên tự chỉnh sửa thêm ở những nội dung khác cho phù hợp với dự án đang thực hiện.

b/ Trong package `com.mycompany.controllers`, tạo class `AdminIndexController`:

```
package com.mycompany.controllers;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller
@RequestMapping("/admin")
public class AdminIndexController {

    @GetMapping("/Home")
    public String adminIndex() {
        return "admin/index";
    }

}
```

b/ Sao chép file `index.html` của admin, dán vào thư mục **admin/product** và đổi tên thành `product.html`, tại box-body, thêm vào một link và một table như sau:

```
<div>

    <a class="label pull-left bg-green" th:href="@{/admin/product-
add}">THÊM HÀNG HÓA MỚI</a>

</div>

<br/>

<table class="table" border="1">

<tr>

    <th>Mã hàng</th>

    <th>Tên hàng</th>
```

```

        <th>Đơn giá</th>
        <th>Xuất xứ</th>
        <th>Loại hàng</th>
        <th>Hình</th>
        <th>Xu hướng</th>
        <th>Phổ biến</th>
        <th>Hành động</th>
    </tr>
    <tr th:each="product:${listPro}">
        <td th:text="${product.mahh}"></td>
        <td th:text="${product.tenhh}"></td>
        <td th:text="${product.dongia}"></td>
        <td th:text="${product.xuatxu}"></td>
        <td th:text="${product.loaihh}"></td>
        <td></td>
        <td th:text="${product.xuhuong}"></td>
        <td th:text="${product.phobien}"></td>
        <td>
            <a class="label pull-right bg-green"
th:href="@{/product/edit/{mahh}}(mahh=${product.mahh})">SỬA</a>
            <a class="label pull-right bg-red"
th:href="@{/product/delete/{mahh}}(mahh=${product.mahh})">XÓA</a>
        </td>
    </tr>
</table>

```

c/ Trong package com.mycompany.controllers, tạo class AdminProductController:

```
package com.mycompany.controllers;

import java.io.IOException;
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.multipart.MultipartFile;
import com.mycompany.models.Image;
import com.mycompany.models.Product;
import com.mycompany.services.Helper;
import com.mycompany.services.ProductService;

@Controller
@RequestMapping("/admin")
public class AdminProductController {

    @Autowired
    private ProductService productService;

    @GetMapping("/product")
    public String adminproduct(Model model) {
        List<Product> lstPro=this.productService.getAll();
```

```

        model.addAttribute("listPro",lstPro);
        return "admin/product/product";
    }
    @GetMapping("/product-add")
    public String admin_product_add() {
        return "admin/product/add-product";
    }
}

```

d/ Trong thư mục admin/product, tạo file add-product.html để tạo form thêm hàng hóa mới, các thẻ tạo form gợi ý như sau:

```

<form role="form" method="post" th:action="@{/admin/product-add}"
enctype="multipart/form-data">
    <div class="box-body">
        <div class="form-group">
            <label for="txtMahh">Mã hàng</label>
            <input type="text" class="form-control" id="txtMahh" placeholder="nhập mã
hàng">
        </div>
        <div class="form-group">
            <label for="txtTenhh">Tên hàng</label>
            <input type="text" class="form-control" id="txtTenhh" placeholder="nhập tên
hàng">
        </div>
        <div class="form-group">
            <label for="txtDongia">Đơn giá</label>
            <input type="text" class="form-control" id="txtDongia" placeholder="nhập
đơn giá">
        </div>
    </div>

```

```

<div class="form-group">
  <label for="txtXuatxu">Xuất xư</label>
  <input type="text" class="form-control" id="txtXuatxu" placeholder="nhập
xuất xư">
</div>

<div class="form-group">
  <label for="selectLoaihang">Chọn loại hàng</label>
  <select name="loaihang" id="selectLoaihang" class="form-control"
required="required">
    <option value="">Giày nam</option>
    <option value="">Giày nữ</option>
    <option value="">Túi xách nữ</option>
    <option value="">Ví nam</option>
    <option value="">Phụ kiện</option>
  </select>
</div>

<div class="form-group">
  <label for="file">Hình sản phẩm</label>
  <input type="file" class="form-control" id="file" name="filename"
placeholder="Chọn file ảnh">
</div>

<div class="xh-pb" style="display:flex; flex-direction:rows;">
<div class="form-group" style="margin-right:20px;">
  <label for="xuhuong">Sản phẩm xu hướng</label>
  <div class="radio">
    <label>
      <input type="radio" name="" id="xuhuong" value="" checked="checked">
Hiện

```

```

        </label>

        <label>

            <input type="radio" name="" id="input" value="" >

                Ân

        </label>

    </div>

</div>

<div class="form-group">

    <label for="phobien">Sản phẩm phổ biến</label>

    <div class="radio">

        <label>

            <input type="radio" name="" id="phobien" value="" checked="checked">

                Hiên

        </label>

        <label>

            <input type="radio" name="" id="input" value="" >

                Ân

        </label>

    </div>

</div>

</div>

</div>

<!-- /.box-body -->

<div class="box-footer">

    <button type="submit" class="btn btn-primary">Thêm mới</button>

</div>

```

</form>

e/ Sao chép class Helper của bài thực hành số 3 vào gói services. Thêm phương thức vào file AdminProductController như sau:

```
@PostMapping("/product-add")

    public String add(@ModelAttribute("product") Product pro,
@RequestParam("filename") MultipartFile f){

        try {

            String fileName = Helper.saveFile("src/main/resources/static/Images", f,
32);

            pro.setHinh(fileName);

            if(this.productService.create(pro)) {

                return "redirect:/admin/product";

            }

        } catch (IOException e) {

            e.printStackTrace();

        }

        return "redirect:/admin/product-add";

    }
```

Kết quả gợi ý cho câu 2:

AdminLTE

Admin Online

Search...

- Quản lý loại hàng FE
- Quản lý hàng hóa FE
- Quản lý người dùng FE
- Quản lý khách hàng FE
- Quản lý banner
- Widgets Hot

Danh mục hàng hóa Thêm, sửa, xóa

Home Examples Blank page

DANH MỤC HÀNG HÓA

THÊM HÀNG HÓA MỚI

Mã hàng	Tên hàng	Đơn giá	Xuất xứ	Loại hàng	Hình	Xu hướng	Phổ biến	Hành động
01001	Men's Office shoes	45	Ý	1		1	0	XOÀ SỬA
02001	Women's Office shoes	52	VN	2		1	1	XOÀ SỬA
02002	Women's Shoes	52	Ấn Độ	2		0	1	XOÀ SỬA
03001	Women's Office bag	45	Pháp	3		1	1	XOÀ SỬA
04001	Men's wallet	47	Pháp	4		0	1	XOÀ SỬA
05002	Dây nịt nam	22	VN	2		0	1	XOÀ SỬA

Footer

Copyright © 2018 Bộ môn Công nghệ phần mềm. Version 2.4.0

CNTT-TNMT

Admin Online

Search...

- Quản lý loại hàng FE
- Quản lý hàng hóa FE
- Quản lý người dùng FE
- Quản lý khách hàng FE
- Quản lý banner
- Widgets Hot

Thêm hàng hóa mới

Home Examples Blank page

Nhập thông tin hàng hóa mới

Mã hàng
nhập mã hàng

Tên hàng
nhập tên hàng

Đơn giá
nhập đơn giá

Xuất xứ
nhập xuất xứ

Chọn loại hàng
Giày nam

Hình sản phẩm
Choose File No file chosen

Sản phẩm xu hướng **Sản phẩm phổ biến**
☒ Hiện ☐ Ẩn
 ☒ Hiện ☐ Ẩn

Thêm mới

Câu 3: sinh viên thực hiện tương tự để có các giao diện quản lý loại hàng, quản lý người dùng, quản lý khách hàng.

--Hết bài thực hành 4--